

A whole program

Everything in this book, used once, on something small and real: a notebook that stores notes in a database, answers from the command line, and comes with its own tests.

Start the project

```
sprfst new notes
cd notes
```

You get:

```
notes/
  project.sprfst
  src/main.spf
  .gitignore
```

`sprfst run .` already works. Now make it do something.

The program

```
~~ notes - storage, commands and tests in one file.
use std.io
use std.db
use std.sys
use std.time
use std.testing

object Notes {
  store: Int = 0
  next_id: Int = 1

  ~~ `:memory:` for tests, a file path for real use
  fn open(self, place: Text) {
    self.store = db.open(place)
    db.exec(self.store, "create table notes (id int, body text, made num, done int
  )")
    let rows = db.query(self.store, "select id from notes order by id desc limit 1
  ")
    let last = rows.first()?.get("id") ?? 0
    self.next_id = last.to_int() + 1
  }

  fn add(self, body: Text) -> Int {
    let id = self.next_id
    db.exec(self.store, "insert into notes values ({id}, '{body}', {time.now()}, 0
  )")
    self.next_id = id + 1
    give id
  }
}
```

```

fn finish(self, id: Int) -> Bool {
    db.exec(self.store, "update notes set done = 1 where id = {id}")
    give true
}

fn open_notes(self) -> [Map<Text, Any>] {
    give db.query(self.store, "select id, body from notes where done = 0 order by
id")
}

fn count(self) -> Int {
    let rows = db.query(self.store, "select count(*) from notes where done = 0")
    give (rows.first()?.get("count") ?? 0).to_int()
}

fn close(self) {
    db.close(self.store)
}
}

test "notes come back in order and can be finished" {
    var n = Notes { }
    n.open(":memory:")
    let first = n.add("buy milk")
    let second = n.add("write chapter thirty")
    testing.same(first, 1, "first id")
    testing.same(second, 2, "second id")
    testing.same(n.count(), 2, "two open")
    n.finish(first)
    testing.same(n.count(), 1, "one open")
    testing.same(n.open_notes().len(), 1, "one listed")
    n.close()
}

fn main() {
    var notes = Notes { }
    notes.open(":memory:")

    let args = sys.args()
    if args.len() > 0 {
        notes.add(args.join(" "))
    } else {
        notes.add("read the guidebook")
        notes.add("write a program")
        notes.add("show someone")
        notes.finish(2)
    }

    io.say("{notes.count()} notes still open")
    for row in notes.open_notes() {
        io.say("  {row.get("id") ?? 0}. {row.get("body") ?? ""}")
    }
    notes.close()
}

```

Run it:

```
sprfst run .  
sprfst run . -- buy bread
```

Everything after `--` reaches the program through `sys.args()`.

Make it keep the notes

One character: give `open` a file instead of `:memory:`.

```
notes.open("notes.db")
```

Ember writes through on every change, so the notes are there tomorrow. Keep `:memory:` in the test and the test stays fast and isolated.

Check your work

```
sprfst test      # the test block above  
sprfst check     # types, without running anything  
sprfst lint      # style and clarity  
sprfst fmt       # layout, in place  
sprfst docs      # docs/ from your `~~` comments
```

A sensible order before committing: `fmt`, then `check`, then `test`.

Grow it

The shape above is the shape of most programs: a type that owns the data, methods that do one thing each, a `main` that reads the world and prints to it, and tests that never touch the real storage.

From here, the rest of the book plugs straight in.

[\225](#)A window instead of a terminal —chapter 21. The state is `notes`, the list is `list { items: ... }`, and `on click` calls `add`.

[\225](#)A web interface —chapter 22. `handle(request)` reads the path and calls the same methods.

[\225](#)A tidy printout —chapter 6, `pad_right` and `join`.

[\225](#)Tags and grouping —chapter 8 and `collections.group_by`.

[\225](#)Something to share —chapter 27, `sprfst` package.

Where to go next

[\22!](#)`examples/` has twenty one complete programs, from `01-hello.spf` to a neural network that learns XOR.

[\22!](#)`std/` is the part of the library written in SPRFST. Read it; it is meant to be read.

`\22!sprfst docs .` writes documentation for whatever you have written, the same way.

`\225`The language, the compiler, the runtime, Forge, the debugger and Studio are all in this repository, in C and Swift and SPRFST. If something here is wrong, it is fixable, and now you know where.

Final exercise

Take the notes program and finish it properly: `add`, `list`, `done` and `search` as real sub-commands read from `sys.args()`, notes kept in a file, a test for each command, and `sprfst package` at the end. That is a complete piece of software, written in a language that did not exist before this book started.